

Session 4.3

SJF and SRTF

Session 4.3: Focus

- **SJF Schedulers**
 - Preemptive and
 - Non-preemptive or SRTF
- **Practice**
 - Problem 1 (non-preemptive)
 - Problem 2 (preemptive)

Schedulers Covered so far ...

- Round-robin Schedulers
 - With time slice (preemptive)
- First-Come-First-Served (FCFS) or FIFO Schedulers
 - Similar to non-preemptive
- Let us see some more different types of schedulers now

SJF (Shortest Job First) Scheduler

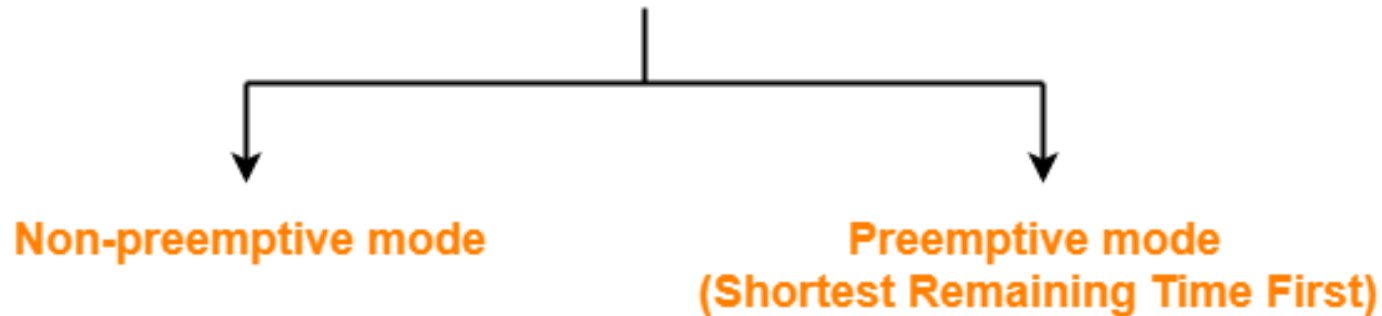


RV
UNIVERSITY

change the world

ATIONAL INSTITUTIONS

Shortest Job First



- Out of all available processes, CPU is assigned to the process having the **smallest burst time**.
- In **case of a tie**, it is broken by **FCFS Scheduling**.
- **SJF Scheduling** can be used in both **preemptive** and **non-preemptive mode**.
- **Preemptive mode** of Shortest Job First is called as **Shortest Remaining Time First (SRTF)**.

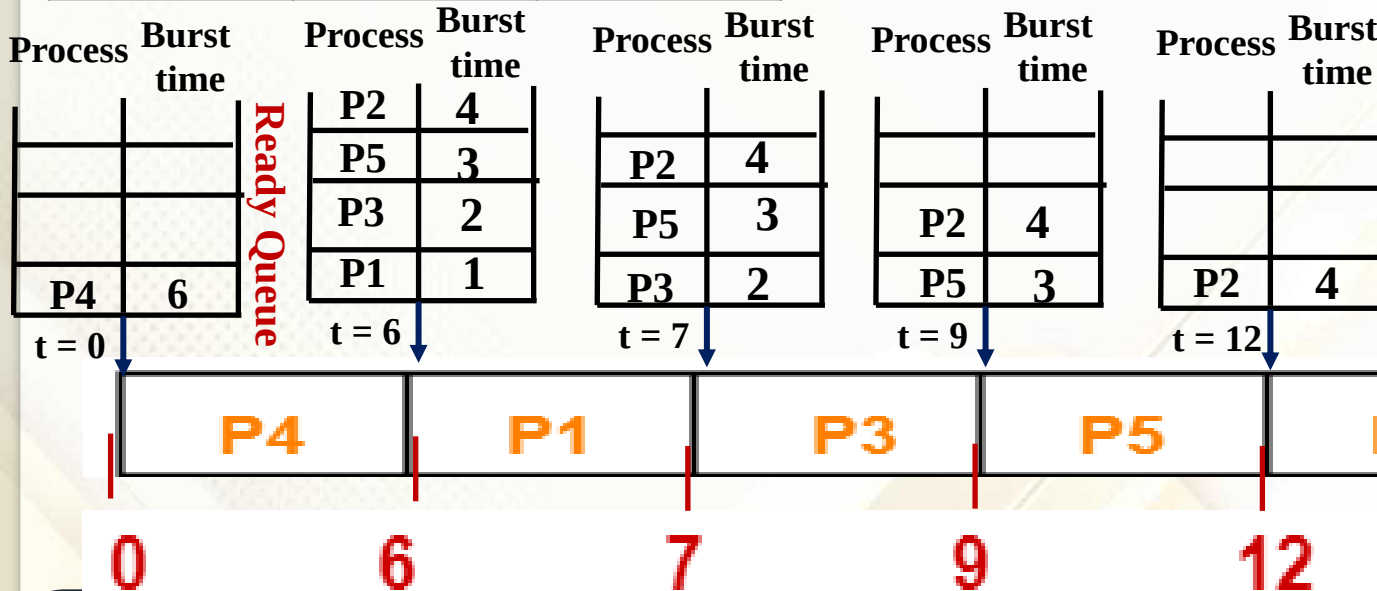
Problem 1: SJF (non-preemptive)

Ref: Tutorial on SJF

Process ID	Arrival Time	Burst Time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

Note that, here, the processes are kept in the ready Q not in the order they arrive, but in their order of burst times, with the lowest being at the head of the Ready Q. This is done by the OS on arrival of any new processes.

Note: In case of processes With the same burst times, the order of arrival is considered and the earlier one is kept ahead.



Note: Since this is a **non-preemptive scheduler**, once the processes start running, the next scheduling will happen only when the running process relinquishes the CPU on its own.

Picture not to scale

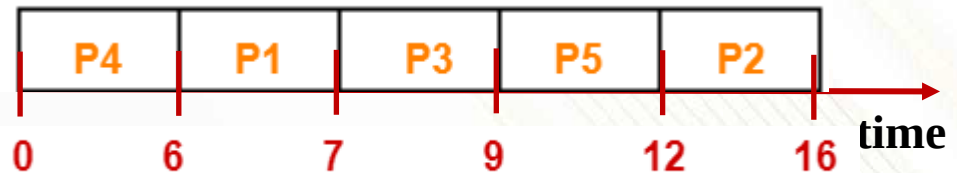
This way of representing processes is called **Gantt chart**

Problem 1: SJF (non-preemptive) contd.

Find **Turnaround time** and **waiting time** of all the processes

Process ID	Arrival Time	Burst Time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

Picture not to scale



Can you give the relationship between them?

Execution time, Turnaround time and Waiting time

Turnaround time = Execution time + Waiting time

Waiting time = Turnaround time – Execution time

Process Id	Exit time	Turn Around time	Waiting time
P1	7	$7 - 3 = 4$	$4 - 1 = 3$
P2	16	$16 - 1 = 15$	$15 - 4 = 11$
P3	9	$9 - 4 = 5$	$5 - 2 = 3$
P4	6	$6 - 0 = 6$	$6 - 6 = 0$
P5	12	$12 - 2 = 10$	$10 - 3 = 7$

Turnaround time = Exit time – arrival time

Problem 2: SJF (Preemptive) - SRTF

Process ID	Arrival Time	Burst Time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

Process	Burst time
P4	5
P5	3
P2	3

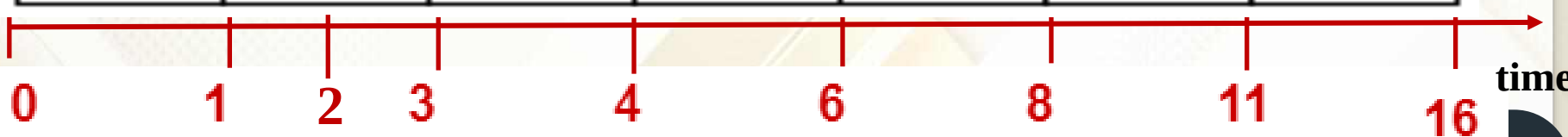
t = 2

SRTF: Shortest Remaining Time First

Note: In case of a **tie**, it is broken by **FCFS** Scheduling.

Process	Burst time	Process	Burst time	Process	Burst time	Process	Burst time
				P4	5	P4	5
				P5	3	P5	3
		P4	5	P2	2	P3	2
P4	6	P2	4	P1	1	P2	2

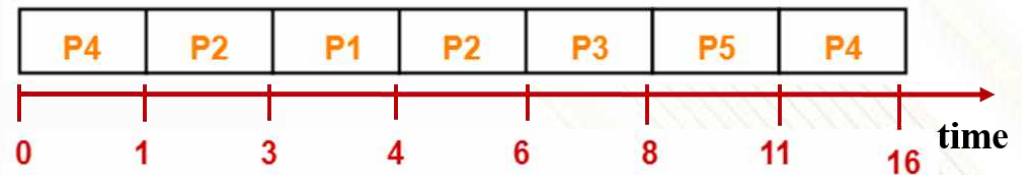
t = 0 t = 1 t = 3 t = 4



Problem 2: SRTF contd ..

Find **Turn around time** and **waiting time** of all the processes

Process ID	Arrival Time	Burst Time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3



Note:

Turn Around time = Exit time – Arrival time

Waiting time = Turn Around time – Burst time

Process Id	Exit time	Turn Around time	Waiting time
P1	4	$4 - 3 = 1$	$1 - 1 = 0$
P2	6	$6 - 1 = 5$	$5 - 4 = 1$
P3	8	$8 - 4 = 4$	$4 - 2 = 2$
P4	16	$16 - 0 = 16$	$16 - 6 = 10$
P5	11	$11 - 2 = 9$	$9 - 3 = 6$

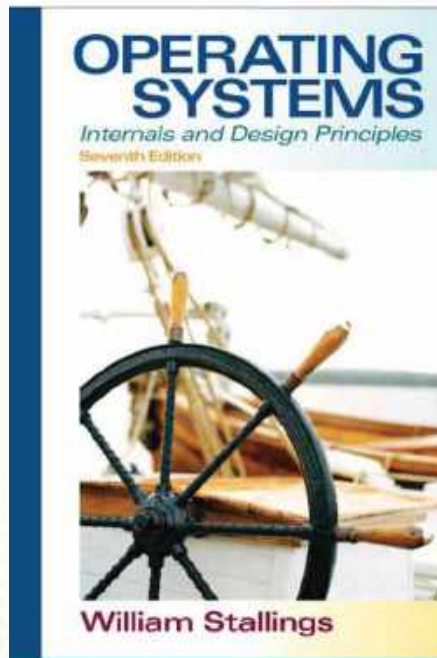
Note that if the **Burst time** is **less** its **waiting time** is also **less** in **SRTF**.

Session 4.3: Summary

- **SJF Schedulers**
 - Preemptive and
 - Non-preemptive or SRTF
- **Practice**
 - Problem 1 (non-preemptive)
 - Problem 2 (preemptive)

References (1 to 5)

Ref 1

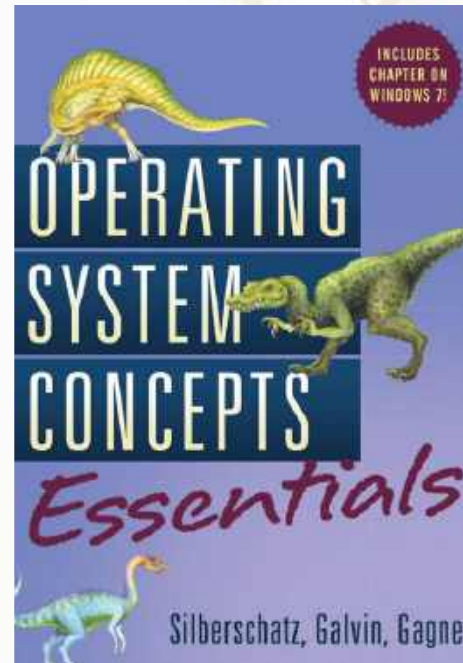


Ref 4

RP2040 Datasheet

A microcontroller
by Raspberry Pi

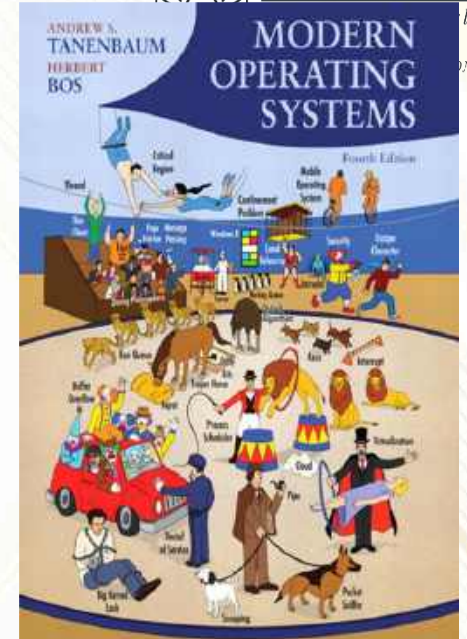
Ref 2



Ref 5

Raspberry Pi Pico C/C++ SDK

Libraries and tools for
C/C++ development on
RP2040 microcontrollers



Id

NS